From Tabucol to Ant Colony Optimization: Transferring Reinforcement-Learning Guidance for Graph Coloring

Abstract. We investigate whether reinforcement-learning guidance trained in a Tabucol pipeline can be transferred to a population-based search and then used to improve the final local search. The study proceeds in four stages. We compare RLTabucol with pure Tabucol and a matched random-policy baseline, diagnose a representative failure with an actor-only experiment, and transfer the frozen policy to ACO through soft pheromone injection. We then pass the converged RLACO solution to Tabucol under the same five-hour refinement budget used by the source baselines. RLTabucol is equal to or better than RLTabucol_0 on eight of eleven instances, while the transfer experiments show that RLACO improves the random-policy baseline on the reported instances and that soft injection is competitive with conflict-based constructed injection. The terminal table shows that RLACO+Tabucol reaches lower color budgets on several difficult instances, including DSJC500.9, `le450_15c`, `le450_15d`, and `le450_25d`. These results support a staged division of labor: the policy supplies structured guidance, ACO supplies population diversity, and Tabucol performs final local repair.

Keywords: Graph coloring · Reinforcement learning · Ant colony optimization · Policy transfer

1 Introduction

Graph coloring is a central combinatorial optimization problem with applications in scheduling, register allocation, and frequency assignment. Classical metaheuristics for graph coloring follow two broad paradigms. Single-solution local search methods, exemplified by Tabucol [**?**], repeatedly modify one coloring through admissible vertex–color moves and use tabu memory to control cycling. Population-based methods, including ant colony optimization (ACO) [**?**], construct multiple colorings and use shared information such as pheromone to balance exploitation with diversification. Both paradigms are effective, but they expose different search mechanisms: Tabucol is well suited to fine local repair, whereas a population-based method can maintain alternative constructions when one incumbent becomes trapped.

Reinforcement learning (RL) has increasingly been coupled with classical combinatorial heuristics rather than used as a complete replacement for them. In this setting, a learned policy proposes actions or construction preferences, and the downstream heuristic refines or exploits them. RLTabucol, introduced

by Frijters [**?**], follows this pattern for graph coloring by interleaving a learned vertex–color recoloring policy with Tabucol local search. We implement the factorized actor–critic policy and evaluate it against a matched random-policy baseline as well as pure Tabucol. Our controlled comparison shows that the learned policy is effective as a source of vertex–color guidance, but not as broadly effective as reported for the original RLTabucol setting. This motivates a more focused diagnostic of where the policy helps and why the direct policy–Tabucol combination can fail.

A policy trained and evaluated only inside its source search framework does not answer whether its guidance reflects the problem or the surrounding algorithm. In particular, a policy trained with Tabucol may encode preferences that are useful for reducing conflicts but poorly matched to the basin that Tabucol can refine from a single incumbent. We therefore use an actor-only experiment on a representative failure instance to separate the policy's coarse conflict reduction from the subsequent local-search repair. This diagnostic provides the bridge from the source-framework comparison to the transfer problem: if the actor still produces structured, low-conflict states when Tabucol is removed, those states may be useful to a different search framework even when the direct handoff is unsuccessful.

The diagnostic also suggests a specific limitation of the training signal. Because the reward design repeatedly emphasizes immediate conflict changes, the policy can be driven toward a low-conflict regime without being explicitly encouraged to generate a diverse set of states that are especially favorable to later Tabucol repair. A direct actor–Tabucol pipeline can consequently revisit a narrow near-feasible basin. ACO offers a complementary mechanism because it constructs a population of alternatives rather than committing to one policy-produced coloring. We therefore first test whether the frozen policy can be transferred to ACO without local-search post-processing, using its per-vertex color distribution as a soft pheromone prior. We then use the resulting RLACO state as the starting point for Tabucol. The terminal RLACO+Tabucol experiment tests the optimization hypothesis that transferable policy guidance and ACO's population diversity can provide Tabucol with a more useful starting state while preserving its strength in final local repair.

The paper develops this argument in a sequence that mirrors the evaluation. We first establish the source-framework behavior with RLTabucol, RLTabucol_0, and pure Tabucol. We then diagnose a failure case with actor-only trajectories. Next, we evaluate cross-framework transfer with RLACO, RLACO_0, and pure ACO, compare soft injection with conflict-based constructed injection, and describe the complementary roles of the RL and ACO phases. Finally, we evaluate RLACO+Tabucol against the three Tabucol-based configurations under the declared two-stage budget. This organization keeps the transfer mechanism, its diagnostic motivation, and the terminal optimization objective connected throughout the paper.

We make four contributions:

1. **A controlled source-framework evaluation.** We implement the RLTabucol pipeline and evaluate the trained policy together with a matched random-policy baseline and pure Tabucol under a common fixed-k protocol. This comparison establishes what can be attributed to the learned vertex–color guidance before introducing a different search framework.
2. **An actor-only diagnosis of the policy–Tabucol interface.** We remove Tabucol on a representative failure instance and compare the resulting actor trajectory with the full pipeline. The diagnostic separates coarse conflict reduction from fine local repair and motivates the need for a more diverse, policy-informed handoff state.
3. **A cross-framework transfer interface and analysis.** We design a soft pheromone-injection interface that converts the policy's per-vertex color distribution into an ACO prior without retraining. We evaluate this transfer against a matched random-policy configuration and pure ACO, compare it with conflict-based constructed injection, and quantify the descriptive shares of the RL and ACO phases.
4. **A terminal RLACO+Tabucol optimization framework.** We pass the converged RLACO state to Tabucol under the same five-hour local-search budget used by the source baselines. This final configuration combines learned guidance, population diversity, and Tabucol repair in one staged optimization procedure, and provides the main end-to-end contribution of the paper.

The remainder of the paper reviews related work, formulates the learned recoloring policy and the search configurations, describes the benchmark and budget protocols, presents the staged results, and discusses limitations and future directions.

2 Related Work

2.1 Local search for graph coloring

Tabu search for graph coloring was introduced by Hertz and de Werra [**?**] and remains a strong single-solution method for fixed-color feasibility. Tabucol repeatedly selects admissible vertex–color moves and uses tabu memory to avoid immediate reversals. Hybrid methods have embedded Tabucol in genetic and adaptive-memory frameworks [**?**,**?**], while surveys have documented the broader range of coloring heuristics [**?**]. These methods are effective at fine local repair, but a single incumbent can have difficulty escaping a narrow near-feasible basin. Our terminal stage uses this strength after a population-based construction stage has supplied a more diverse starting state.

2.2 Ant colony optimization for coloring

ACO approaches construct colorings incrementally from pheromone and heuristic information. Costa and Hertz [**?**] adapted the construction paradigm to graph

coloring, and later work has combined ACO with prediction or learned heuristics [**?**,**?**]. These approaches generally learn or tune guidance inside the target ACO framework. Our setting is different: the policy is trained with Tabucol and transferred without retraining. The comparison with pure ACO isolates the value of adding the transferred prior, while the $\text{RLACO}_{\text{config}}$ comparison isolates the choice of a soft distributional interface.

2.3 Learning-guided combinatorial search

RL has been used to guide coloring through self-play, value-based methods, graph neural networks, and learned ordering heuristics [**?**,**?**,**?**,**?**,**?**]. Hybrid RL–local-search methods similarly use a policy to propose moves that a classical heuristic refines [**?**,**?**]. Most evaluations remain within the framework used for training. The present study treats cross-framework transfer as the object of evaluation: it asks whether vertex–color preferences learned for local search remain useful when converted into a pheromone prior for construction.

The resulting contribution is a staged interaction between the two paradigms. The RL phase supplies learned preferences and coarse conflict reduction, ACO supplies population diversity, and Tabucol supplies fine repair. The source-framework, transfer, injection, phase-share, and terminal experiments are designed to distinguish these roles rather than collapse them into one aggregate score.

3 Methodology

This section presents the learned policy and the staged evaluation used in the study. We first describe the shared actor–critic policy and the RLTabucol source framework, including pure Tabucol and the random-policy control. We then define the actor-only diagnostic that examines failure cases, followed by the RLACO transfer experiment, its injection controls, and the phase-share analysis. Finally, we describe the RLACO+Tabucol terminal configuration, which tests whether policy transfer and ACO diversity improve the starting state for Tabucol optimization. Figure 1 summarizes these linked configurations.

3.1 Learned recoloring policy

The core of all evaluation configurations is a learned policy that selects vertex–color recoloring actions. The coloring task is modeled as a finite-horizon Markov decision process with horizon I. At each step, the state contains $G = (V, E)$, the color budget k, and the current coloring c_t. An action is $a_t = (v_t, q_t)$. If $H_v(s, q)$ denotes the number of neighbors of v assigned color q, the implementation gives a valid recoloring the immediate reward

$$r_t^{\text{imm}} = H_{v_t}(s_{t-1}, s_{t-1}(v_t)) - H_{v_t}(s_t, q_t), \tag{1}$$

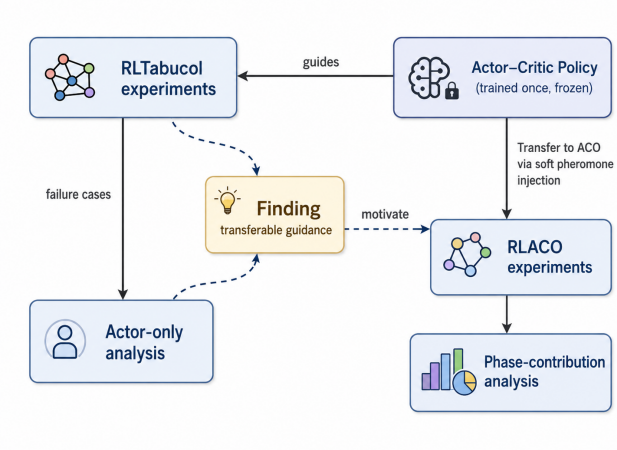


Fig. 1. Overview of the source, diagnostic, transfer, and terminal configurations. The learned actor–critic policy is trained once in RLTabucol, frozen, and reused throughout.

and gives zero reward for a no-op. After the fixed RL phase, the Tabucol call contributes a terminal reward equal to the input conflict count minus the conflict count of its returned state. Thus the episode return combines the per-step conflict-reduction rewards with the final Tabucol reward; it is not a single global $f(c_t) - f(c_{t+1})$ expression at every transition.

The policy is implemented as an actor–critic architecture. The actor network factorizes the joint action into two components: a vertex-selection distribution and a conditional color-selection distribution:

$$\pi_{\theta,\phi}((v,q) \mid s) = \pi_\theta^{\mathrm{v}}(v \mid s)\, \pi_\phi^{\mathrm{c}}(q \mid s, v). \tag{2}$$

The vertex-selection head π_θ^{v} is a two-layer multilayer perceptron ($3 \to 16 \to 1$ with ReLU) applied independently to every candidate vertex, followed by a softmax over all vertices. It uses three vertex-level features: degree, conflicting edge count, and number of distinct neighbor colors. The color-selection head π_ϕ^{c} is a separate two-layer MLP ($3 \to 16 \to 1$ with ReLU) applied to every vertex–color pair, followed by a softmax over colors. It uses three color-level features: number of conflicting neighbors, class size, and total vertex count. Because each MLP processes only local features without graph-level pooling, the actor network is permutation-equivariant by construction through local vertex features, and generalizes across graph sizes and color budgets.

The critic network shares the same feature extraction structure as the actor and appends a linear layer over the flattened $n \times k$ logit matrix to produce a scalar state value $V_\psi(s)$. This value is used to compute advantage estimates for Proximal Policy Optimization (PPO) [**?**], which updates both the actor and critic parameters using the standard clipped surrogate objective. During RLTabucol training, the reward combines immediate per-step conflict-reduction

rewards with the final reward returned by Tabucol. The default policy is trained on $G(n{=}250, p{=}0.5)$ random graphs with $k{=}24$ colors. We use this checkpoint because it is the common frozen source policy for the reported transfer and terminal experiments and is trained on the default random-graph distribution before DIMACS evaluation. Once trained, the policy parameters are frozen and reused across the source, transfer, and terminal configurations without further updates. Detailed PPO and training hyperparameters are reported in Appendix A.

3.2 The RLTabucol pipeline

We implement the RLTabucol pipeline by interleaving the learned policy with Tabucol local search within an episodic loop [**?**,**?**]. Each run begins with a random k-coloring. Within each episode, the actor performs I sequential recoloring steps, producing an intermediate coloring that is then passed to Tabucol. Tabucol is a single-solution local search that iteratively selects the best admissible vertex–color recoloring move, subject to a tabu list that prevents recently reversed moves. It runs for at most T iterations and returns both a refined coloring and the cumulative reward accumulated by its internal moves. The detailed Tabucol procedure is given in Algorithm 3 in Appendix A.

The coloring carried into the next episode is selected probabilistically: with probability β the current coloring is kept, and with probability $1-\beta$ the best coloring found during that Tabucol call is returned instead. Algorithm 1 summarizes the complete RLTabucol pipeline. During training, this loop runs for 50 epochs and the collected trajectories are used for PPO updates. During testing, the actor parameters are fixed and the loop continues until a zero-conflict coloring is found or the five-hour wall-clock budget is exhausted.

To verify the effectiveness of the RLTabucol framework and the trained policy, we define a matched baseline called RLTabucol_0 that replaces the trained actor with a uniform random policy over the same action space. All other components—episode structure, reward computation, Tabucol procedure, return rule, and time budget—remain identical.

3.3 Actor-only diagnostic

The RLTabucol experiment shows that the policy provides useful guidance but still has failure cases. We therefore define an actor-only configuration that removes Tabucol entirely. In this setting, each episode consists solely of I RL recoloring steps, and the running-best score is tracked across episodes without any local search refinement. The diagnostic is applied to `le450_15d` at $k{=}15$, a persistent failure case for the default policy. By comparing the actor-only trajectory with the full RLTabucol pipeline, we ask whether the failure comes from the policy itself, from Tabucol's repair capacity, or from their interaction: the policy may provide a low-conflict regime that is poorly matched to Tabucol, while Tabucol may return a state that gives the next policy episode little useful structure.

Algorithm 1 The RLTabucol pipeline

Input: G, k, actor $\pi_{\theta,\phi}$, steps I, Tabucol iters T, prob. β, budget $\mathcal{B}$
$c \leftarrow$ random k-coloring; $c^\star \leftarrow c$
while $\mathcal{B}$ not exhausted **do**
  for $t = 0, \ldots, I-1$ **do**
    $(v, q) \sim \pi_{\theta,\phi}(\cdot \mid (G, k, c))$; $c \leftarrow c[v \mapsto q]$
    if $f(c) < f(c^\star)$ **then**
      $c^\star \leftarrow c$
    end if
    if $f(c^\star) = 0$ **then**
      return $c^\star$
    end if
  end for
  $c \leftarrow \textsc{Tabucol}(G, k, c, T, \beta)$
  if $f(c) < f(c^\star)$ **then**
    $c^\star \leftarrow c$
  end if
  if $f(c^\star) = 0$ **then**
    return $c^\star$
  end if
end while
return $c^\star$

3.4 Transfer to ACO via soft pheromone injection

Having observed that the policy can provide useful guidance in RLTabucol on several instances, we ask whether the same policy can also benefit a different major class of search algorithms, namely population-based construction methods such as ACO. The RLACO pipeline tests this by using the policy's color-selection distribution to bias the pheromone matrix of an ant colony optimization procedure. This transfer experiment is a controlled test of whether learned guidance from a local-search-based framework can benefit a population-based, construction-based framework without retraining. Because the source policy is not uniformly superior to the baselines, this stage establishes transferability and the mechanism used by the later optimization experiment.

The ACO component follows a MAX–MIN ant system [**?**] with pheromone clipping, elite reinforcement, a pseudo-random-proportional construction rule, and no ACO-internal local search. An ant constructs a coloring by visiting vertices one by one and selecting colors according to the standard construction rule that combines pheromone and heuristic information. The detailed ACO procedure is given in Algorithm 4 in Appendix A.

The key interface that enables the RLACO study is the soft pheromone injection, which we design to bridge the actor's output and the ACO construction process. Given the current state s, the color-selection head of the actor produces $P(v, q) = \pi^{\mathrm{c}}_{\phi}(q \mid s, v)$, representing the actor's preference for assigning color q to vertex v. As a softmax output, $P(v, \cdot)$ is normalized over colors, i.e., $\sum_{q=1}^{k} P(v, q) = 1$ for every vertex v. At the beginning of each ACO call, after

Algorithm 2 The RLACO pipeline

Input: G, k, actor $\pi_{\theta,\phi}$, steps I, ACO iters T_{ACO}, ants m, budget B
Params: $\tau_0, \tau_{\min}, \tau_{\max}, \sigma$
$c \leftarrow$ random k-coloring; $\quad c^\star \leftarrow c$
while elapsed $< B$ **do**
  for $t = 0, \ldots, I-1$ **do**
    $(v, q) \sim \pi_{\theta,\phi}(\cdot \mid s(c)); \quad c \leftarrow c[v \mapsto q]$
    if $f(c) < f(c^\star)$ **then**
      $c^\star \leftarrow c$
    end if
  end for
  $P(v, q) \leftarrow \pi^{\mathrm{c}}_{\phi}(q \mid s(c), v)$
  $\tau \leftarrow \textsc{SoftInject}(P, \tau_0, \tau_{\min}, \tau_{\max}, \sigma)$
  $c \leftarrow \mathrm{ACO}(G, k, c, \tau, m, T_{\mathrm{ACO}})$
  if $f(c) < f(c^\star)$ **then**
    $c^\star \leftarrow c$
  end if
  if $f(c^\star) = 0$ **then**
    return $c^\star$
  end if
end while
return $c^\star$

the RL phase, the pheromone matrix is reset to the uniform base level τ_0, and the policy distribution is injected through a soft interpolation:

$$\tau(v, q) \leftarrow \tau_0 + \big(\min(\tau_{\max},\, \sigma\tau_0) - \tau_0\big)\, P(v, q), \tag{3}$$

where $\sigma > 1$ controls the injection strength. The resulting pheromone values are immediately clipped to $[\tau_{\min}, \tau_{\max}]$. The actor therefore provides a soft prior rather than a deterministic assignment: every ant still constructs a coloring from scratch using the biased pheromone, while the RL handoff coloring is retained only as the incumbent passed to ACO. After each ACO iteration, standard evaporation and elite reinforcement are applied, followed by clipping to the same bounds. The policy thus affects ACO only through the initial pheromone matrix and the incumbent at the start of each ACO call. Algorithm 2 summarizes the complete RLACO pipeline.

We also evaluate $\mathrm{RLACO}_{\mathrm{config}}$, called conflict-based constructed injection in the results. It uses the same RL phase but replaces the learned color distribution with a normalized prior inversely related to the number of conflicting neighbors assigned each candidate color. This control tests whether soft policy transfer contributes beyond a generic conflict-based prior. Pure ACO removes the RL phase, and RLACO_0 uses the matched random-policy phase.

ACO local-search post-processing is deliberately disabled in RLACO. This design choice serves two purposes. First, it isolates the effect of the pheromone prior: with local search enabled, any observed difference between RLACO and RLACO_0 could be attributed to the local search amplifying small initial dif-

ferences rather than to the policy's guidance itself. Second, it keeps RLACO a controlled transfer stage rather than a complete solver comparison. The end-to-end optimization question is tested separately by passing the converged RLACO state to Tabucol.

RLACO_0 is a matched baseline that replaces the trained actor with a random policy while preserving the same RL steps, handoff mechanism, ACO procedure, and time budget. Comparing RLACO with RLACO_0 tests whether the trained policy provides more useful guidance than a random-policy prior under the same ACO pipeline.

To understand whether the total improvement in each RLACO episode comes mainly from the RL phase, mainly from the ACO phase, or from both, we record the conflict count at three checkpoints: before the RL phase (f_e^{start}), after the RL phase (f_e^{RL}), and after the ACO phase (f_e^{ACO}). The per-episode contribution of each phase is:

$$\Delta_e^{\text{RL}} = \left[f_e^{\text{start}} - f_e^{\text{RL}}\right]_+, \qquad \Delta_e^{\text{ACO}} = \left[f_e^{\text{RL}} - f_e^{\text{ACO}}\right]_+, \tag{4}$$

where $[x]_+ = \max(x, 0)$ ensures that a phase receives zero credit if it increases conflicts. Aggregating across all episodes yields cumulative contributions S_{RL} and S_{ACO}, from which the relative RL share is:

$$\text{share}^{\text{RL}} = \frac{S_{\text{RL}}}{S_{\text{RL}} + S_{\text{ACO}}}, \qquad \text{share}^{\text{ACO}} = 1 - \text{share}^{\text{RL}}. \tag{5}$$

This decomposition helps characterize how the transferred policy participates in the search: whether it directly drives the improvement, or whether its main role is to provide a favorable starting point that the downstream ACO phase then exploits.

3.5 RLACO+Tabucol terminal optimization

The terminal configuration tests whether a transferable policy and ACO population diversity provide Tabucol with a more useful starting state than the direct RLTabucol interaction. RLACO is first run with the instance-dependent transfer time used in the transfer experiments. Its converged incumbent is then passed directly to Tabucol without retraining or an additional RL phase. Tabucol receives a separate five-hour wall-clock budget, matching the budget used by pure Tabucol, RLTabucol, and RLTabucol_0.

The terminal score is the final best conflict count, and a run is successful only when this score is zero. We compare RLACO+Tabucol with all three Tabucol-based configurations. The pure-Tabucol comparison tests whether the staged framework improves the heuristic baseline; the RLTabucol and RLTabucol_0 comparisons test whether transfer and population diversity improve over the two source-policy configurations. The objective is optimization on the tested color budgets, rather than a claim of universal success or global optimality.

Table 1. DIMACS benchmark instances used in the experiments. The column $\chi/k^\star$ reports the known chromatic number χ when available and the reference color number $k^\star$ used in the experimental protocol. The size group is based on the number of vertices: Small for $n \leq 300$, Medium for $300 < n \leq 500$, and Large for $n > 500$.

Instance	$\chi/k^\star$	Size	n	Family
`DSJC250.5`	?/28	Small	250	Random, density 0.5
`DSJC500.1`	?/12	Medium	500	Random, density 0.1
`DSJC500.5`	?/47	Medium	500	Random, density 0.5
`DSJC500.9`	?/126	Medium	500	Random, density 0.9
`flat300_26_0`	26/26	Small	300	Flat graph
`flat300_28_0`	28/28	Small	300	Flat graph
`flat1000_76_0`	76/81	Large	1000	Flat graph
`le450_15c`	15/15	Medium	450	Leighton graph
`le450_15d`	15/15	Medium	450	Leighton graph
`le450_25c`	25/25	Medium	450	Leighton graph
`le450_25d`	25/25	Medium	450	Leighton graph

4 Experimental Setup

4.1 Benchmark instances

The experiments are conducted on instances from the DIMACS graph coloring benchmark suite. These instances are widely used for evaluating graph coloring algorithms and cover several graph families. The DSJC instances are random graphs, where the suffix indicates the edge density. The `le450` instances are Leighton graphs with structured coloring constraints. The flat instances are quasi-random graphs generated with a prescribed target number of colors and relatively balanced vertex degrees. Table 1 lists the benchmark instances used in the experiments. The second column reports the pair $\chi/k^\star$, where χ denotes the chromatic number when it is known ("?" indicates that it is unknown) and $k^\star$ is the number of colors in the best known solution.

4.2 Experimental environment and evaluation metrics

Detailed RLTabucol, RLACO, and PPO parameter settings, including the Tabucol, ACO, RLTabucol, and RLACO procedures, are reported in Appendix A. Parameters not explicitly overridden in the implementation follow the Tianshou PPO defaults for the library version used in the experiments. Training is performed exclusively on Erdős–Rényi random graphs using a T4 GPU; no DIMACS instance is seen during training. Testing is conducted on CPU. All reported running times are wall-clock times.

For the RLTabucol experiment, both RLTabucol and its random-policy baseline RLTabucol_0 are evaluated with ten independent runs per instance, each with a five-hour wall-clock time limit. This budget is chosen as a common convention for DIMACS graph coloring benchmarks. The main metric is the smallest

number of colors k for which a legal coloring ($f(c) = 0$) is found within the budget, denoted k_{best}. When two methods reach the same k, running time and success rate serve as secondary indicators. The success count reports the number of zero-conflict runs out of ten.

Pure Tabucol removes the RL phase while retaining the same Tabucol solver, tested color budgets, ten-run protocol, and five-hour wall-clock allowance.

For the RLACO transferability experiment, both RLACO and its random-policy baseline RLACO_0 are evaluated under the fixed target color budget listed in Table 1. The main metric is the running-best conflict value at the end of each run. As noted in Section 3.4, ACO local-search post-processing is deliberately disabled so that RLACO serves as a transferability test rather than a complete solver. This metric is used because finding the smallest solvable color number for each instance would require many pilot runs over different k values. Using a fixed target k and the final conflict count avoids this cost while still showing whether the trained policy reduces conflicts more effectively than a random policy. We report the observed best (lowest final running-best across ten runs) and the mean±standard deviation. The RLACO experiments use instance-specific time budgets, chosen based on instance size and difficulty; for each instance, RLACO and RLACO_0 are allocated essentially the same wall-clock budget, so the comparison remains paired at the instance level. Because both the RL phase and the ACO phase are stochastic, each configuration is evaluated with ten independent runs and we report both the observed best and the mean±standard deviation to account for run-to-run variability. In addition to RLACO_0 and pure ACO, we evaluate $\text{RLACO}_{\text{config}}$ using conflict-based constructed injection with the same RL and ACO budgets; this control tests the soft learned prior against a conflict-based prior.

For the terminal RLACO+Tabucol experiment, RLACO is first run for the instance-dependent transfer time used in the RLACO experiments, with the policy frozen and without additional training. Its converged incumbent is then passed to Tabucol, which receives a separate five-hour wall-clock budget. This Tabucol budget is the same as that used by pure Tabucol, RLTabucol, and RLTabucol_0, so the terminal comparison preserves the local-search allowance while testing the value of the transferred, population-explored starting state. The terminal metric is the final best conflict count, and a run is successful only when the score equals zero. The terminal table follows the supplied results document; three selected raw groups contain nine summaries rather than ten, and missing observations are not counted as failures.

5 Results

5.1 Learned guidance in the source RLTabucol framework

Table 2 reports the three source-framework configurations in one view. The most direct policy comparison is RLTabucol against the random-policy baseline RLTabucol_0, because the two share the same episodic structure, Tabucol solver,

Table 2. Source-framework comparison among pure Tabucol, RLTabucol, and the random-policy baseline RLTabucol$_0$. Time reports best / mean wall-clock seconds over successful runs only. Bold k_{best} values are the lowest per row, including ties. Best and mean times are compared and bolded independently only among methods attaining the row-minimum color budget.

Instance	Pure Tabucol			RLTabucol			RLTabucol$_0$		
	k_{best}	Success	Time (s)	k_{best}	Success	Time (s)	k_{best}	Success	Time (s)
DSJC250.5	**28**	10/10	14.6 / 45.5	**28**	10/10	243.8 / 3046.5	**28**	10/10	**2.6** / **269.0**
DSJC500.1	**12**	7/10	2356.2 / 6001.4	**12**	8/10	**936.9** / **3616.9**	13	10/10	0.4 / 0.6
DSJC500.5	**49**	10/10	**408.5** / **1519.6**	**49**	6/10	2375.5 / 7664.3	50	10/10	101.9 / 1303.2
DSJC500.9	**128**	8/10	**2513.0** / **8521.6**	**128**	4/10	3062.7 / 8749.3	130	7/10	1080.5 / 4289.1
le450_15c	16	10/10	0.2 / 45.2	16	10/10	4.1 / 246.7	**15**	10/10	**273.1** / **1283.2**
le450_15d	16	9/10	0.3 / 302.5	16	10/10	32.2 / 1085.3	**15**	9/10	**932.4** / **6956.8**
le450_25c	**26**	9/10	5.4 / 12.4	**26**	10/10	44.7 / 141.0	**26**	10/10	**3.3** / **16.5**
le450_25d	**26**	10/10	7.6 / 14.5	**26**	10/10	44.8 / 106.6	**26**	10/10	**1.9** / **17.5**
flat300_26_0	**26**	10/10	**8.8** / **215.8**	**26**	10/10	229.2 / 1147.8	**26**	10/10	134.3 / 704.4
flat300_28_0	**30**	1/10	**9453.8** / **9453.8**	31	10/10	150.4 / 1278.2	**30**	1/10	14021.1 / 14021.1
flat1000_76_0	88	1/10	16485.4 / 16485.4	**87**	1/10	**8920.2** / **8920.2**	90	10/10	303.9 / 1349.2

and five-hour testing budget; only the policy is changed. RLTabucol reaches a lower k_{best} on four instances, ties on four, and requires more colors on three. The gains include two-color improvements on DSJC500.9 and a three-color improvement on flat1000_76_0. The learned policy therefore contains useful search information, but its benefit inside the source pipeline is clearly instance-dependent rather than uniformly positive.

The pure-Tabucol column in the same table asks a different question. Both methods receive the same five-hour wall-clock allowance, but pure Tabucol can devote essentially the whole budget to local search, whereas RLTabucol spends part of the same budget executing the actor and the repeated policy–Tabucol interaction. Even with that distinction, the optimization objective is stronger than merely showing that the policy is non-random: the combined method should eventually improve on a strong Tabucol baseline. Under the present source coupling, it does not do so consistently. RLTabucol lowers k_{best} only on flat1000_76_0 (87 versus 88), ties pure Tabucol on nine instances, and is worse on flat300_28_0 (31 versus 30). At tied color budgets, the timing and success results are also mixed. For example, on DSJC500.1 RLTabucol reaches $k = 12$ in 8/10 runs and has a lower best and mean successful-run time than pure Tabucol, whereas on DSJC500.5 it succeeds in only 6/10 runs at $k = 49$ compared with 10/10 for pure Tabucol.

This distinction matters for the rest of the experiments. The RLTabucol–RLTabucol$_0$ comparison shows that the trained actor is not equivalent to random recoloring, while the comparison with pure Tabucol shows that the source framework does not yet convert that information into a consistently stronger optimizer. The remaining question is therefore not simply whether the policy or Tabucol is weak in isolation, but whether the way they interact limits the value of the learned guidance.

5.2 Diagnosing the policy–Tabucol interface

Figure 2 examines `le450_15d` at $k = 15$. Across ten complete actor-only runs, the best conflict counts at termination range from 203 to 243, with mean 225.5 and median 223.5. The ten full-pipeline diagnostic runs finish between 5 and 21 conflicts, with mean 12.7 and median 13.5 (rounded to 14 in the figure). Removing Tabucol therefore leaves the actor far from feasibility, even though the actor produces substantial coarse conflict reduction. Once Tabucol is restored, the search moves from hundreds of conflicts into a near-feasible regime, but none of these runs reaches zero.

The diagnostic narrows the likely failure mode. The actor is not useless: it can substantially reduce the initial conflict count. Tabucol is not ineffective either: it provides most of the fine-grained repair needed to move from the actor-only plateau to the 5–21 range. What remains is the handoff between the two. A low-conflict state is not necessarily a state from which Tabucol can complete the final repair, and the source pipeline repeatedly propagates a single incumbent from one episode to the next.

The training signal makes this interpretation plausible, although it does not establish it causally. Each episode contains 300 RL transitions with immediate conflict-reduction feedback at every step. The first 299 transitions contain only that immediate signal, while the final transition also includes the reward returned by the downstream Tabucol call. Thus, information about Tabucol's response is delayed relative to the dense local feedback available throughout the RL phase. Repeated policy decisions may consequently favor a relatively narrow class of locally attractive, low-conflict states without learning equally well which of those states are easiest for Tabucol to repair. In that sense, the single-incumbent pipeline may provide insufficient diversity near feasibility. We use this as a working explanation and design motivation, not as a measured diversity result.

This motivates a different use of the same policy. Rather than asking one policy-produced trajectory to supply the state that Tabucol must repair, the next experiment transfers the learned color preferences to a population-based construction process that can generate multiple policy-biased alternatives.

5.3 Cross-framework transfer and the role of policy guidance

Table 3 evaluates the transferred policy at fixed target color budgets with ACO-internal local search disabled. $\text{RLACO}_{\text{soft}}$ obtains lower observed best and mean conflicts than both pure ACO and RLACO_0 on all ten instances. Relative to RLACO_0, the observed-best reduction ranges from 20 conflicts on `le450_25c` to 154 on `DSJC500.9`. Two-sided Welch tests compare the ten final conflict values of RLACO_0 and $\text{RLACO}_{\text{soft}}$ separately for each instance; all ten differences remain significant after Holm correction, with a largest adjusted p-value of 0.00122.

The RLACO_0 comparison should be read as a configuration-level random-policy reference rather than a perfectly matched soft-prior ablation. The random policy does not provide a learned per-vertex color distribution, so the tested

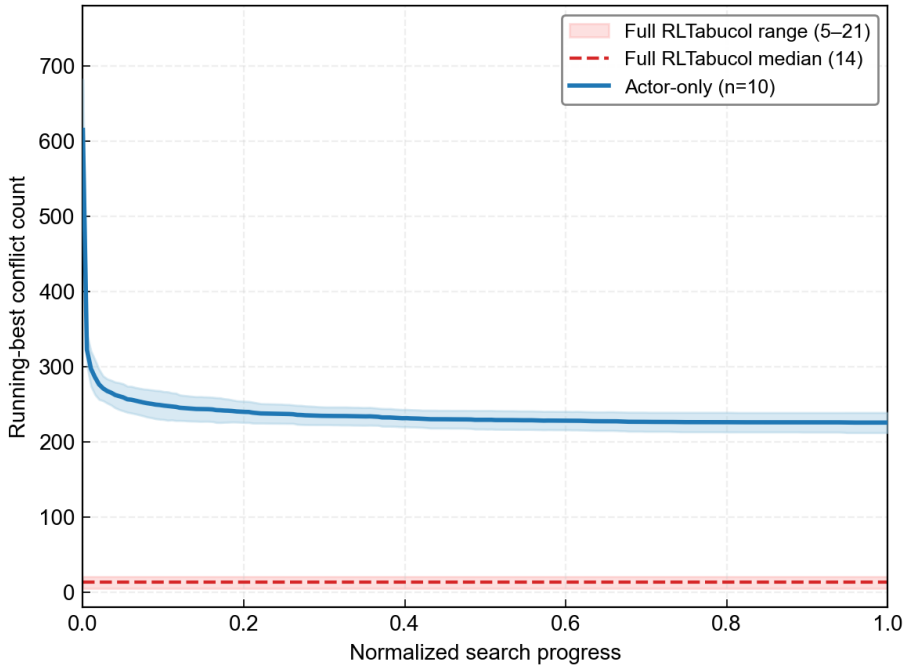


Fig. 2. Actor-only diagnostic on `le450_15d` ($k = 15$). The actor-only curve summarizes ten runs over normalized search progress; the red band shows the full RLTabucol final range of 5–21 conflicts. Its median of 13.5 is rounded to 14 in the legend.

baseline transfers its realized random coloring through hard pheromone injection. Pure ACO supplies a second reference with no RL guidance at all. Taken together, these controls show that the complete learned-transfer configuration provides a consistent advantage over both unguided ACO and a random-policy transfer pipeline, without attributing the entire gap to a single isolated factor.

The three source-framework failure cases are useful here. At the fixed transfer budgets, the adverse learned-policy effect seen in RLTabucol disappears: $\text{RLACO}_{\text{soft}}$ reduces the observed best from 150 to 58 relative to RLACO_0 on `flat300_28_0`, from 295 to 222 on `le450_15c`, and from 290 to 229 on `le450_15d`. These are not yet legal colorings, so this stage should not be described as recovery. It does show, however, that the usefulness of the same frozen policy changes when its information is consumed by a different search framework.

$\text{RLACO}_{\text{config}}$ provides a more specific control over the injection interface. It retains the trained RL phase and the same soft-injection pipeline, but replaces the actor's color probabilities with a normalized conflict-based prior. $\text{RLACO}_{\text{soft}}$ achieves a lower observed best on six instances, ties on `DSJC500.9`, and is worse on three. Its mean is lower on seven instances and higher on the same three losses. The losses are modest in two cases: the observed-best difference is one conflict

Table 3. Transfer comparison at fixed tested k, without ACO-internal local search. Entries are best conflicts (mean±sample standard deviation); the lowest best value per row is bold, including ties. All cells contain ten complete runs. RLACO_0 uses a random-policy phase followed by hard injection of the resulting coloring, and therefore serves as a random-policy transfer reference rather than a matched soft-prior ablation. Welch t uses RLACO_0 minus $\text{RLACO}_{\text{soft}}$, with ten runs per group; two-sided p-values are Holm-adjusted across ten instances.

Instance	$\chi/k^\star$	k	Pure ACO	RLACO_0 (hard)	$\text{RLACO}_{\text{config}}$	$\text{RLACO}_{\text{soft}}$	Welch t
DSJC250.5	?/28	28	69 (74.6±3.5)	70 (76.6±2.9)	21 (24.7±2.8)	**16 (22.7±3.1)**	39.7
DSJC500.1	?/12	12	123 (134.2±6.1)	136 (141.4±2.3)	57 (65.7±6.9)	**48 (62.9±11.6)**	21.0
DSJC500.5	?/47	48	224 (229.9±3.1)	225 (230.0±3.6)	76 (84.7±6.9)	**74 (78.1±3.4)**	97.9
DSJC500.9	?/126	126	192 (194.9±2.8)	196 (202.1±3.0)	**42 (48.4±3.9)**	**42 (47.4±3.4)**	110.3
flat300_26_0	26/26	26	179 (187.8±5.0)	187 (194.7±3.6)	**86 (92.9±4.9)**	93 (96.7±4.9)	50.9
flat300_28_0	28/28	28	136 (145.5±4.8)	150 (155.6±2.6)	**57 (62.3±4.1)**	58 (62.4±3.1)	73.4
le450_15c	15/15	15	293 (297.6±2.4)	295 (298.0±1.6)	**221 (237.0±10.7)**	222 (244.0±13.1)	24.9
le450_15d	15/15	15	295 (300.8±2.6)	290 (301.4±5.3)	234 (254.5±13.6)	**229 (249.2±13.2)**	11.6
le450_25c	25/25	25	68 (70.3±1.7)	63 (68.0±2.7)	60 (62.9±2.4)	**43 (57.2±6.9)**	4.7
le450_25d	25/25	25	70 (72.2±1.4)	66 (70.0±1.7)	61 (64.7±2.4)	**44 (50.5±6.4)**	9.2

on flat300_28_0 and le450_15c. By contrast, the learned prior improves the observed best by 17 conflicts on both le450_25c (43 versus 60) and le450_25d (44 versus 61), and by 9 on DSJC500.1 (48 versus 57). The learned prior therefore has the stronger overall empirical profile, but it does not universally dominate the explicit conflict heuristic.

Figure 3 helps explain why the transferred guidance need not act in the same way on every instance. On the random and flat graphs, the RL phase accounts for 91.2%–99.3% of the cumulative positive conflict reduction. The share falls to 72.1% and 66.7% on le450_15c and le450_15d. The pattern changes sharply on le450_25c and le450_25d: the RL shares are only 0.45% and 0.84%, so almost all recorded positive reduction occurs during ACO.

Those last two instances are informative because $\text{RLACO}_{\text{soft}}$ is still strong there despite contributing almost no direct positive reduction during the RL phase. This suggests two operating modes for the transferred policy. On some graphs it acts mainly as a direct reducer of conflicts; on others, its more important role appears to be shaping the distribution from which ACO constructs and reinforces solutions. Immediate RL conflict reduction is therefore not a necessary condition for useful transfer. The phase shares remain descriptive, not causal: negative changes are clipped to zero, and the ACO phase acts on a state produced by the preceding RL phase. Their value is in showing that the same learned guidance can be exploited differently by the downstream search.

Because the soft learned prior consistently improves on the random-policy transfer reference, remains competitive with the explicit conflict prior, and produces several substantially larger gains, we retain soft policy injection for the final optimization experiment.

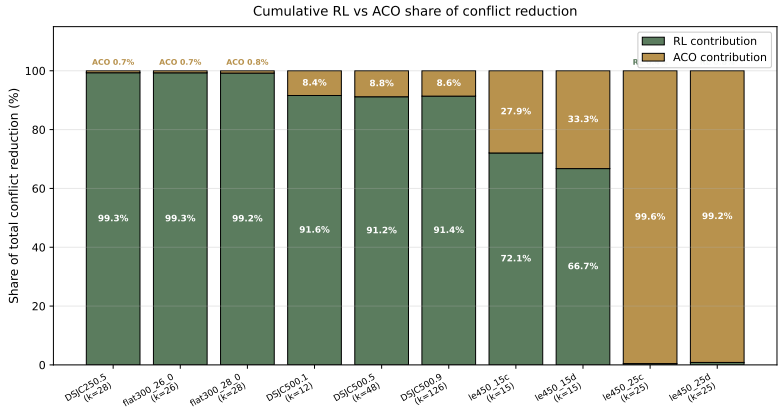


Fig. 3. Cumulative shares of positive conflict reduction recorded during the RL and ACO phases across ten transfer instances. Shares are descriptive and are not a causal decomposition of the final performance difference.

5.4 Optimization with RLACO-initialized Tabucol

The final experiment asks whether the transferred construction stage can produce a starting state that a strong local solver can exploit. We denote this configuration by RLACO→Tabucol to emphasize the direction of the handoff: RLACO is run first using the same instance-specific preparation budget as in the transfer experiment, and its converged incumbent is then supplied to a separate Tabucol run. The reported solving time starts at this handoff and measures downstream Tabucol refinement; RLACO initializer-generation time is outside this timing metric by design.

The primary result is the attainable color threshold. RLACO→Tabucol reaches a lower k_{best} than pure Tabucol on five instances and ties it on the other six. Relative to RLTabucol it again records five wins and six ties, and the same 5–6–0 win–tie–loss profile holds against RLTabucol$_0$. It is therefore best or tied for best on all eleven instances. Two cases are strictly better than all three comparators: `DSJC500.9`, where it reaches $k = 126$ instead of 128, 128, and 130, and `le450_25d`, where it reaches $k = 25$ instead of 26 for all three source methods.

The three instances on which the trained source pipeline underperformed its random-policy baseline are all recovered at the terminal stage. On `le450_15c` and `le450_15d`, RLACO→Tabucol reaches $k = 15$ rather than the RLTabucol value of 16; on `flat300_28_0`, it reaches $k = 30$ rather than 31. The `DSJC500.9` result gives the clearest link across the two transfer stages: RLACO$_{\text{soft}}$ first reduces the fixed-$k = 126$ residual to an observed best of 42 conflicts without local search, after which the separate Tabucol stage finds a legal 126-coloring. The source methods reach only 128, 128, and 130 colors under their respective protocols.

Table 4. Terminal optimization comparison. Each method reports its attained k_{best}, the number of successful runs out of ten, and best / mean time in seconds over successful runs only. Bold k_{best} values are the lowest per row, including ties. Best and mean times are compared and bolded independently only among methods attaining that row-minimum color budget; times from methods reported at larger k are shown for completeness but are not used for cross-method speed claims. For RLACO→Tabucol, timing starts at the RLACO-to-Tabucol handoff and excludes RLACO initializer-generation time.

Instance	Pure Tabucol			RLTabucol			RLTabucol_0			RLACO→Tabucol		
	k_{best}	Success	Time (s)	k_{best}	Success	Time (s)	k_{best}	Success	Time (s)	k_{best}	Success	Time (s)
DSJC250.5	**28**	10/10	14.6 / 45.5	**28**	10/10	243.8 / 3046.5	**28**	10/10	2.6 / 269.0	**28**	9/10	**1.5** / **36.5**
DSJC500.1	**12**	7/10	2356.2 / 6001.4	**12**	8/10	**936.9** / **3616.9**	13	10/10	0.4 / 0.6	**12**	9/10	1358.0 / 4994.4
DSJC500.5	**49**	10/10	**408.5** / **1519.6**	**49**	6/10	2375.5 / 7664.3	50	10/10	101.9 / 1303.2	**49**	10/10	424.0 / 1755.7
DSJC500.9	128	8/10	2513.0 / 8521.6	128	4/10	3062.7 / 8749.3	130	7/10	1080.5 / 4289.1	**126**	1/10	**7330.6** / **7330.6**
le450_15c	16	10/10	0.2 / 45.2	16	10/10	4.1 / 246.7	**15**	10/10	273.1 / 1283.2	**15**	2/10	**130.8** / **252.0**
le450_15d	16	9/10	0.3 / 302.5	16	10/10	32.2 / 1085.3	**15**	9/10	932.4 / 6956.8	**15**	1/10	**578.5** / **578.5**
le450_25c	**26**	9/10	5.4 / 12.4	**26**	10/10	44.7 / 141.0	**26**	10/10	3.3 / 16.5	**26**	10/10	**1.5** / **7.4**
le450_25d	26	10/10	7.6 / 14.5	26	10/10	44.8 / 106.6	26	10/10	1.9 / 17.5	**25**	2/10	**8480.7** / **8771.6**
flat300_26_0	**26**	10/10	**8.8** / 215.8	**26**	10/10	229.2 / 1147.8	**26**	10/10	134.3 / 704.4	**26**	10/10	13.3 / **167.9**
flat300_28_0	**30**	1/10	9453.8 / 9453.8	31	10/10	150.4 / 1278.2	**30**	1/10	14021.1 / 14021.1	**30**	1/10	**4788.2** / **4788.2**
flat1000_76_0	88	1/10	16485.4 / 16485.4	**87**	1/10	**8920.2** / **8920.2**	90	10/10	303.9 / 1349.2	**87**	1/10	13323.9 / 13323.9

Success counts need to be interpreted at the color budget being attempted. A method that reaches a lower k is solving a harder fixed-k problem, so its success fraction is not directly comparable with a competitor reported at a larger color budget. At the new frontiers, success is understandably sparse: RLACO→Tabucol succeeds in 1/10 runs at $k = 126$ on `DSJC500.9` and at $k = 15$ on `le450_15d`, and in 2/10 runs on `le450_15c` and `le450_25d`. At shared targets the picture is stronger. On `DSJC500.1` at $k = 12$, it succeeds in 9/10 runs, compared with 8/10 for RLTabucol and 7/10 for pure Tabucol; on `DSJC500.5` at $k = 49$, it reaches 10/10, matching pure Tabucol and exceeding RLTabucol's 6/10.

The downstream timing results add a second view of initialization quality. Among methods that attain the row-minimum k_{best}, RLACO→Tabucol has the shortest best successful-run refinement time on seven of eleven instances and the shortest mean successful-run time on eight. Two of those rows, `DSJC500.9` and `le450_25d`, are cases in which RLACO→Tabucol alone reaches the best color budget. Excluding those two rows gives a stricter matched-target view: it remains fastest on five of nine instances by best successful time and six of nine by mean successful time. These means are conditional on success. In particular, when only one run succeeds, the reported best and mean are necessarily identical and should not be read as evidence of stable runtime.

Taken together, the terminal results show that the transferred initializer is useful in two ways. It pushes the attainable color threshold downward, which is the primary optimization objective, and it often gives Tabucol a state that can be repaired efficiently once that state is available. The latter statement is about downstream refinement rather than equal-total-compute runtime, because the RLACO preparation stage is not included in the reported terminal solving time.

6 Discussion and Limitations

The experiments separate two questions that are easy to conflate: whether the policy has learned useful information, and whether the framework in which it was trained makes good use of that information. The RLTabucol–RLTabucol$_0$ comparison answers the first question more directly. The trained policy improves k_{best} on four instances and ties on four, including relatively large gains on `DSJC500.9` and `flat1000_76_0`; it is therefore not behaving like a random recoloring rule. Yet the comparison with pure Tabucol is much less favorable: one improvement, nine ties, and one loss in k_{best}. Poor source-pipeline performance is therefore not the same as poor policy quality. The more useful interpretation is that policy value is framework-dependent, while some of the information learned by the policy can still be reusable outside that framework.

The actor-only diagnostic gives a concrete reason to examine the interface. The actor can remove a large number of conflicts on its own, but it stalls hundreds of conflicts from feasibility on `le450_15d`. Tabucol then supplies the fine repair needed to reach the 5–21 conflict range, yet the combined pipeline still fails to finish. This is consistent with a handoff problem rather than a simple failure of either component. A good handoff state is not defined only by a low value of $f(c)$; it must also lie in a region that the downstream local search can exploit.

The reward structure offers one plausible explanation for why the source handoff can be imperfect. Immediate conflict feedback is available at every RL transition, whereas the information returned by Tabucol appears only at the final transition of a 300-step episode. With the PPO discount factor $\gamma = 0.99$, that delayed signal is also discounted when credit is propagated toward earlier actions. This does not show that the Tabucol reward is numerically dominated, nor does it prove that the reward design causes the observed failures. It does make the credit-assignment problem asymmetric: the policy receives dense information about how to reach lower-conflict states and much sparser information about which low-conflict states are especially repairable by Tabucol.

The search dynamics may reinforce the same effect. RLTabucol propagates one returned coloring from episode to episode, so repeated policy decisions and Tabucol calls evolve a single incumbent trajectory. The policy actions and the β return rule are stochastic, so the pipeline is not deterministic. Still, it may repeatedly revisit a relatively narrow family of low-conflict structures. Such states may lack sufficient diversity for the final repair, or may simply occupy unfavorable Tabucol basins. We cannot distinguish those two possibilities from the present experiments because no explicit diversity or basin-distance metric is measured. The diversity explanation should therefore be read as a mechanism hypothesis, not as a causal finding.

ACO changes this interface without retraining the policy. Instead of requiring the actor trajectory to produce the one coloring that the next solver must repair, soft injection turns the actor's per-vertex color distribution into a population-level construction bias. Each ant still builds a coloring from scratch. This matters because the transferred information is preserved as a preference distribution while the downstream search is free to realize that preference in multiple, differ-

ent constructions. In this role, ACO is not a replacement for the policy; it is a mediator that changes how the policy's information is expressed.

The transfer controls support this interpretation, but they also set a useful boundary on what can be claimed. RLACO_0 uses a random-policy trajectory and hard injection of the resulting coloring, so it is a random-policy transfer reference rather than an exact soft-prior ablation. Its large gap to $\text{RLACO}_{\text{soft}}$ is therefore configuration-level evidence. Pure ACO provides an independent no-policy reference, and $\text{RLACO}_{\text{config}}$ supplies the more focused prior comparison by retaining the trained RL phase and replacing only the learned soft prior with an explicit conflict-based one. Across these three controls, the learned-transfer configuration has the strongest overall profile: it beats pure ACO and RLACO_0 on all ten tested instances and wins more often than it loses against the conflict prior, while the conflict prior remains better on several cases. The evidence supports the learned distribution as a useful transfer interface; it does not establish universal superiority over every alternative prior.

The phase analysis also shows that transfer is not synonymous with direct RL improvement. On the random and flat instances, most recorded positive conflict reduction occurs during the RL phase. On `le450_25c` and `le450_25d`, however, the RL share is below 1%, even though the soft configuration remains strong in the fixed-k comparison. The policy can therefore be useful in at least two ways: by directly reducing conflicts, or by changing the distribution presented to the downstream construction process. The latter is especially relevant to cross-framework transfer, because it shows that the policy need not act as a competent standalone solver to provide search-relevant information. The phase shares themselves are descriptive; they do not causally attribute the final gap between configurations.

The RLACO→Tabucol experiment converts that transferred information into the main optimization result. Its k_{best} is best or tied for best on all eleven instances, with five wins and six ties against each of the three comparators and no losses in the observed color threshold. The strictest gains occur on `DSJC500.9` and `le450_25d`, where it reaches a lower color budget than pure Tabucol, RLTabucol, and RLTabucol_0 alike. It also recovers all three cases in which the trained source pipeline was worse than its random-policy baseline. These results are stronger than the fixed-k transfer experiment because the policy-guided construction is no longer judged only by residual conflicts; the downstream Tabucol stage converts the transferred state into legal colorings at equal or lower color budgets.

The timing results point in the same direction, with an important qualification. Once the RLACO initializer is available, the downstream Tabucol stage records the shortest best successful-run time on seven of eleven rows and the shortest mean successful-run time on eight; after removing the two rows where it alone reaches the best color budget, the corresponding counts are five of nine and six of nine. This suggests that the transferred initializer is often easier for Tabucol to exploit, not merely capable of reaching a harder k. At the same time, the means are computed only over successful runs, so they must be read along-

side the success count. A 1/10 result describes a rare successful trajectory, not stable runtime behavior.

The terminal timing protocol is intentionally downstream-facing. RLACO is used as an initializer-generation stage, and the reported solving clock starts when that state is handed to Tabucol. Producing the initializer can require several hours under the instance-specific convergence budgets used in the transfer experiment, so the present results should not be interpreted as an equal-total-compute speed comparison. The trade-off is explicit: additional preprocessing is used to obtain a starting state that can improve the attainable color threshold and, in many cases, reduce the subsequent Tabucol refinement time. Reducing that preparation cost would make the approach more attractive when end-to-end wall-clock efficiency is the primary objective.

Several limitations remain. First, the proposed explanation based on restricted state diversity and handoff quality is not directly measured; future work could compare solution distances, trajectory concentration, or local-search basin statistics. Second, the delayed-credit explanation is consistent with the reward structure but has not been isolated by a reward-design ablation. Third, the random-policy transfer reference uses hard rather than soft injection, so the strong $\mathrm{RLACO}_{\mathrm{soft}}$–$\mathrm{RLACO}_0$ gap should remain a configuration-level comparison; the pure-ACO and conflict-prior controls partly compensate for this limitation but do not remove it. Finally, the study uses one frozen final policy, one soft-injection strength, a moderate DIMACS test set, and ACO as the only cross-framework transfer target. These constraints do not alter the observed results, but they limit how broadly the mechanism should be generalized.

7 Conclusion

We studied a learned vertex–color policy across two search frameworks and a terminal hybrid. RLTabucol is competitive with its random-policy baseline on most instances, but the actor-only diagnostic shows why direct handoff to Tabucol can fail: coarse conflict reduction does not provide enough diversity for the final local repair.

Soft transfer to ACO addresses this mismatch. RLACO improves the reported random-policy and pure-ACO references, and the injection comparison supports retaining the policy's full color distribution rather than using only a conflict-based constructed coloring. The phase-share analysis shows that RL and ACO play complementary roles.

The main contribution is RLACO+Tabucol. After instance-dependent RLACO convergence, the transferred state is refined by Tabucol under the same five-hour budget used by the source baselines. The terminal results improve the best color budget on DSJC500.9, le450_15c, le450_15d, and le450_25d, and reach the best or tied-best budget on the other listed instances, although several cells have low success counts. The evidence supports using policy transfer to create diverse, policy-informed starting states for a strong downstream local search.

Future work should complete the missing terminal seeds, evaluate fixed-budget protocols, and test the mechanism with additional policies, injection strengths, and target metaheuristics.

A Experimental details

A.1 Algorithm pseudocode

Algorithms 3 and 4 give the pseudocode for the Tabucol and ACO procedures referenced in Sections 3.2 and 3.4.

Algorithm 3 Tabucol procedure

Input: $G = (V, E)$, colors k, coloring c_0, limit T
$c \leftarrow c_0$; $c^\star \leftarrow c_0$; init tabu list
for $t = 1, \dots, T$ **do**
  $(v, q) \leftarrow$ best admissible move (v conflicting, $q \neq c(v)$)
  Add $(v, c(v))$ to tabu list; $c \leftarrow c[v \mapsto q]$
  if $f(c) < f(c^\star)$ **then**
    $c^\star \leftarrow c$
  end if
  if $f(c^\star) = 0$ **then**
    return $c^\star$
  end if
end for
return $c^\star$ w.p. $1 - \beta$; else c

Algorithm 4 ACO procedure

Input: $G = (V, E)$, k, ants m, iters T, pheromone τ
$c^\star \leftarrow$ random k-coloring
for $t = 1, \dots, T$ **do**
  for each ant $a = 1, \dots, m$ **do**
    Construct c_a via pseudo-random-proportional rule
    if $f(c_a) < f(c^\star)$ **then**
      $c^\star \leftarrow c_a$
    end if
  end for
  Evaporate; reinforce elite; clip τ to $[\tau_{\min}, \tau_{\max}]$
  if $f(c^\star) = 0$ **then**
    return $c^\star$
  end if
end for
return $c^\star$

A.2 Parameter settings

Tables 5–7 report the RLTabucol, RLACO, and PPO settings used in the experiments.

Table 5. RLTabucol parameters.

Parameter	Training	Testing
RL steps I	300	300
Init coloring	Uniform rand.	Uniform rand.
Max epoch	50	No limit
Tabucol iters T	5000	10000
Tabu tenure A, α	10, 1.2	10, 1.2
Return prob. β	0.2	0.2
Run count	PPO sched.	10
Time budget	No limit	18000s

Table 6. RLACO parameters.

Parameter	Value
Target k	Fixed (Tbl. 1)
Ants m	40
ACO iters T_{ACO}	50
$\alpha_{\mathrm{ACO}}, \beta_{\mathrm{ACO}}$	1.0, 3.0
Evaporation ρ	0.20
Q, q_0	1.0, 0.85
Elite, weight	3, 2.0
$\tau_0, \tau_{\min}, \tau_{\max}$	1.0, 0.05, 8.0
Injection σ	8.0
ACO local search	Disabled

Table 7. PPO training parameters.

Parameter	Value
Optimizer, LR	Adam, $3{\times}10^{-4}$
Clip ϵ	0.2
Buffer, batch	30000, 512
Envs, test eps	10, 10
Steps/epoch	50000
Steps/collect	2000
Repeat/collect	10
γ, entropy	Tianshou defaults
Value-loss coef.	Tianshou defaults